ZPDPatch: Evaluating Trajectory-Conditioned Targets for Execution-Guided Program Repair

***Abstract*—We present ZPDPATCH, a trajectory-supervised repair technique that mines execution improvements as training targets and selects generated programs by execution. Program repair for programming exercises benefits from correct peer solutions, learned transformations, and execution feedback. Submission histories offer an additional signal: earlier unsuccessful revisions can distinguish users who reach the same current program. However, a history-aware system can improve simply by generating more candidates, and changing its target rule also changes its training population. Establishing a trajectory-specific benefit therefore requires separate controls for target construction, input history, and candidate breadth. ZPDPATCH mines three evidence relations, trains independent policies, and selects a validation-frozen portfolio with an optional edit budget. We evaluate 997 Seen and 250 problem-held-out Python inputs, replicate matched-target training on 181 student-held-out Java trajectories, and separately examine completed development runs. Matched terminal targets repair 19.2 and 11.2 percentage points more Python inputs than progress targets; progress-target repairs retain 5.6 and 6.5 more token points on joint successes, and Java reproduces both directions. Three fixed candidate draws alone add 15.1 Seen points, while the completed history-specific improvements fail their fixed controls. The mixed portfolio repairs 61.5%/74.8% of Seen/Unseen inputs, versus 72.0%/81.2% for the retained current-only terminal triple; retrieval-based generation reaches 80.2% on Seen.**

***Index Terms*—automated program repair, submission trajectories, target construction, execution-guided repair**

I. INTRODUCTION

Automated program repair (APR) generates changes to a failing program and validates them against an executable specification [1]–[4]. For programming exercises, a repair can provide code-specific feedback beyond an online judge's verdict [5]–[7]. Earlier submissions additionally record attempted strategies, repeated errors, and tests already passed. Should a repair depend on this path to the current code, rather than on that code alone?

Existing methods cluster, align, or normalize correct peer programs [8]–[10], or use learned generation, retrieval, diagnostics, and repair conversations [6], [7], [11]–[14]. These are strong alternatives: LSGen repairs 80.2% of our Seen inputs. Their success nevertheless does not establish that earlier submissions add information beyond current code, adaptation, and additional generation opportunities.

We define *trajectory-conditioned repair* as repair whose supervision or generated change depends on the sequence leading to the current submission. Two users may reach similar code after different failed revisions, suggesting different next changes. The Zone of Proximal Development (ZPD) motivates support informed by prior activity [15]–[18]. Our testable hypothesis is narrower: authentic history should improve executed repair over current-code, shuffled-history, and other-user controls without increasing regressions or unresolved outcomes. An observed sequence does not measure mastery, and a smaller patch does not establish learning.

A second use of a trajectory changes the training target: later improvements can replace the eventual accepted solution. Our terminal rule emits 40,454 examples, whereas the progress rule emits 21,416, changing both targets and source frequencies. Candidate breadth also matters: three fixed stochastic draws add 15.1 Seen repair-rate points over the expected single draw. A gain over one terminal adapter therefore cannot isolate relational supervision or authentic history. We use source-matched retraining, same-target independent seeds, and matched candidate budgets to distinguish these explanations.

We present ZPDPATCH: *Mine* constructs terminal, strict-verdict, and test-progress relations; *Train* fits independent policies; and *Select* freezes a validation-selected checkpoint triple. An execution gate returns a repair or retains current code, optionally within an edit budget. Separate controls replace targets with matched peer programs, remove or perturb histories, and hold draws fixed. These controls are not unreported ingredients of the deployed mixed portfolio.

On 997 Seen and 250 problem-held-out Python inputs, matched terminal targets repair 19.2 and 11.2 points more inputs, while progress targets retain 5.6 and 6.5 more token points on joint repairs. Student-disjoint Java retraining reproduces both directions. However, Mixed reaches 61.5%/74.8%, below the current-only terminal triple's 72.0%/81.2%. Subsequent development improves generic execution selection but yields no candidate satisfying every authentic-history adoption condition. We retain the existing version: the evidence supports a target-induced coverage–locality tradeoff, not the proposed history-specific advantage.

Contributions. We summarize our contributions as follows:

- **Problem and hypothesis:** We distinguish trajectory-derived targets from authentic-history conditioning and candidate breadth, and formulate controls that can reject a history-specific explanation even when aggregate repair coverage increases.
- **Technique:** We specify Mine, Train, and Select, including the three retained-submission rules, exact validation portfolio selection, and an execution gate with explicit fallback and edit-budget semantics.
- **Evaluation:** We separate target, provenance, decoder, checkpoint, and history effects on Python and Java, report the completed development failures under their original

criteria, and provide a claim–evidence map to the retained experimental artifacts.

II. Preliminaries

A. Submissions and Execution Evidence

For a problem p, a submission state is a tuple $s = (c, v, o)$, where c represents complete source code, v denotes the recorded online-judge verdict, and o maps each available testcase identifier to its re-executed outcome. The problem context d_p contains the statement and resource limits, and $\mathcal{E}_p$ is the available test suite. A trajectory $\tau = [s_1, \ldots, s_n]$ orders submissions by one user on one problem. The canonical dataset ends each trajectory at its first recorded acceptance; this endpoint is the final accepted submission within the retained trajectory, not a later post-acceptance revision. Recorded verdicts and replayed outcomes remain distinct fields, because runtime versions and execution variability can change a program's observed behavior.

A training example pairs input $x = (d_p, h_i)$ with later code y, where h_i is the permitted submission prefix ending at the current state. A target constructor determines both the retained prefix and its target. At inference, the current program is always available; a history-conditioned prompt also includes earlier observed submissions, whereas a current-only prompt does not. The future accepted program is a training target or an offline assessment reference, never part of the inference prompt. A *terminal target* is the accepted endpoint. A *progress target* is the next retained state under an execution-evidence rule and can itself be accepted. We reserve *intermediate target* for a retained nonterminal state; the Progress dataset is not restricted to such states.

B. Generate-and-Validate Selection

Algorithm 1: Execution-Gated Repair

Input: Current program (c), context (x), ordered generators (S), tests ($\mathcal{E}$), edit budget (B).
Output: Returned program (c^*).

1 $c^* \leftarrow c$; $q^* \leftarrow$ Execute$(c, \mathcal{E})$;
2 **foreach** $v \in S$ **do**
3 $z \leftarrow$ Generate(v, x);
4 **if** $B < \infty$ *and* $\mathrm{ted}^\dagger(c, z) > B$ **then**
5 **continue**;
6 $q \leftarrow$ Execute$(z, \mathcal{E})$;
7 **if** $q = 1$ **then**
8 **return** z;
9 **if** $(q, -\mathrm{ted}^\dagger(c, z)) > (q^*, -\mathrm{ted}^\dagger(c, c^*))$ **then**
10 $c^* \leftarrow z$; $q^* \leftarrow q$;
11 **return** c^*;

Algorithm 1 describes the common controller. For now, we set aside how the generators S are trained and selected. Line 1 executes the current code and makes it the fallback. Each generator emits a complete candidate, which is excluded if it violates a finite edit budget. Execution then determines whether to accept immediately or update the best partial repair. The final return preserves the fallback when no candidate improves its score. Here q is the fraction of tests passed, and $\mathrm{ted}^\dagger$ is AST tree edit distance when both changed programs parse, zero for unchanged code, and infinity otherwise. An unrestricted budget bypasses the distance check, allowing syntax repair; a finite budget rejects changes whose source distance is undefined.

This controller exposes three independent decisions: the source of generators, their input context, and the number of generation opportunities. Reference-based repair changes generation by supplying aligned peer programs [8]–[10]. Neural and language-model repair changes it through training and prompting [6], [7], [19], [20]. Iterative methods also update context with execution feedback [12], [13]. These are not all supervised-learning methods and need not share a target constructor. Our comparisons isolate specific implementations rather than assuming that all prior systems train on accepted endpoints.

C. Limitation and Goal

Consider three wrong-answer submissions with pass vectors $(1, 0, 0)$, $(1, 1, 0)$, and $(0, 1, 1)$, followed by an accepted state. The second preserves every previously passed test and gains another; the third gains a test but loses the first. Verdict-only filtering cannot distinguish them, while test-wise progress retains the second but not the third relative to it. Terminal supervision instead pairs each failed source independently with the acceptance. This example illustrates different training signals, not evidence that the intermediate change is smaller or easier for a learner.

TABLE I
Canonical training rules before matched controls. Counts are examples, not unique sources or test successes.

Rule	Target and context	Examples
Answer	Each non-AC source alone to the accepted endpoint	40,454
Strict	Retained prefix to the next better-verdict state	16,973
Progress	Retained prefix to the next verdict/test-progress state	21,416

Table I shows why exchanging a target rule is not a one-variable intervention: example counts and source frequencies change together. The relation-level comparison characterizes complete policies, whereas matched-source retraining estimates target replacement on a common population. A scoped audit of eight directly related educational-repair studies also motivates explicit candidate accounting: five evaluate or deploy multiple generation opportunities, and three of those five lack a matched single-opportunity quantity [7], [12]–[14], [21]. This is a purposive evidence map, not a systematic review or a claim that adaptive iterations equal independent draws. Its coding rules remain in the artifact.

These observations motivate constructing auditable trajectory-based supervision and determining whether it improves repair beyond current-code and budget-matched

alternatives. Correctness, source preservation, and history specificity are separate outcomes. A smaller patch does not satisfy a correctness objective, and higher pooled correctness alone does not satisfy a history-specific objective. The canonical evaluation measures these dimensions separately; later development additionally requires non-increasing regressions and unknown outcomes under fixed controls.

Algorithm 2: ZPDPATCH: Mine, Train, Select

Input: Training trajectories (H), cached evidence (O), validation inputs (U), seeds (Σ), portfolio size (k), edit budget (B).
Output: Frozen ordered checkpoint portfolio (S).

```
/* Step 1. Mine                                          */
(D_A, D_S, D_P) ← (∅, ∅, ∅);
foreach τ ∈ H do
    D_A ← D_A ∪ TerminalPairs(τ);
    foreach ρ ∈ {S, P} do
        R ← [s_1];
        foreach s_t ∈ [s_2, ..., s_n] do
            if Improves_ρ(R[−1], s_t; O) then
                D_ρ ← D_ρ ∪ {(R, c_t)};
                R ← R ‖ [s_t];
/* Step 2. Train                                         */
V ← ∅;
foreach ρ ∈ {A, S, P} and σ ∈ Σ do
    V ← V ∪ {Train(D_ρ, σ)};
/* Step 3. Select                                        */
foreach v ∈ V do
    R_v^B ← Validate(v, U, B);
S ← Select({R_v^B}_{v∈V}, k);
return FreezeAndOrder(S) ;          // input to Algorithm 1
```

III. Our Approach

A. Overview

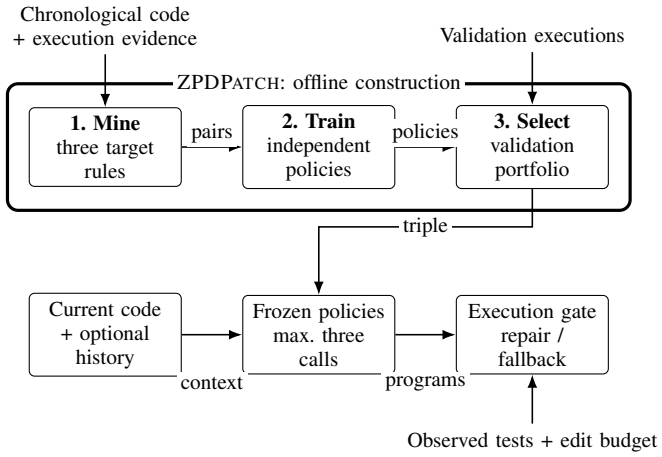


Fig. 1. Overview of ZPDPATCH. Training and selection are offline; deployment uses frozen generators. Matched-target and history controls instantiate separate training/input variants, not extra inference stages.

Figure 1 separates offline construction from online repair. Mine constructs relation-specific datasets, Train fits adapters, and Select freezes a validation-selected portfolio. Each checkpoint receives the same permitted input and generates independently; candidates never enter another checkpoint's prompt. Algorithm 1 validates outputs and retains a fallback. The mixed pool contains three seeds per relation; Answer-only and current-only variants are controls. The final recommendation, current-only Answer-3Seed, is an evaluated comparator, not a renamed history model.

B. Main Algorithm

Algorithm 2 returns a checkpoint portfolio, not a repaired test program. It mines terminal pairs and retained scans, trains one adapter per relation–seed combination, and searches validation subsets before freezing their order. Deployment uses Algorithm 1. The primary configuration uses three seeds and $k = 3$; breadth and pool controls explicitly vary these opportunities.

C. Mine

The goal of Mine is to make every supervision pair traceable to an observed source and a later submission. Submissions are grouped by problem and user, chronologically ordered, and cut at the first acceptance. Canonical normalization removes consecutive duplicates while preserving the source code used for generation and execution. The builder imposes no maximum history or transition count. A whole-trajectory filter instead excludes a trajectory before splitting if any required prompt–target configuration exceeds 4,096 tokens. Exclusion changes the evaluated population; it never cuts off a program or history and presents the fragment as complete.

The coarse severity function assigns 0 to accepted (AC), 1 to wrong answer (WA), 2 to time or memory limit exceeded (TLE/MLE), 3 to runtime error (RE), and 4 to compile error (CE); unrecognized and internal outcomes occupy the most severe category. This is an operational ordering, not semantic distance to correctness. Strict compares recorded verdicts. Progress additionally compares replayed outcomes when the recorded verdict strings agree. Its cache must be complete, and compared vectors must have equal, nonempty testcase keys. Provenance includes problem, code, tests, and execution configuration. Infrastructure uncertainty in later development is reported as unknown, not converted into a favorable repair label.

For equal testcase universes, $o_t \prec_\times o_r$ holds when every test in o_t has severity no greater than its counterpart in o_r, with a strict decrease on at least one. The improvement predicates are:

$$\begin{aligned} \mathsf{Improves}_S(r,t) &\iff \mathrm{sev}(v_t) < \mathrm{sev}(v_r), \\ \mathsf{Improves}_P(r,t) &\iff \mathsf{Improves}_S(r,t) \\ &\quad \vee\ (v_t = v_r \wedge o_t \prec_\times o_r). \end{aligned} \tag{1}$$

Intuitively, Equation (1) retains coarse improvements and progress hidden by an unchanged verdict. A severity improvement such as RE to WA need not increase the pass fraction, and a verdict-improving edge does not require test-wise dominance. The mined relation supplies supervision, not a deployment-safety guarantee.

Answer emits an independent pair for each non-AC source and the accepted endpoint. Thus $[s_1, \ldots, s_9]$ yields $([s_1], c_9), \ldots, ([s_8], c_9)$; it does not expose accumulated prefixes to Answer training. Strict and Progress start with $R = [s_1]$, compare each later state with the *last retained* state, and append it only when the predicate holds. Before appending s_t, the builder emits (R, c_t). For WA, improved WA, AC, Strict emits $([s_1], c_3)$, whereas Progress emits $([s_1], c_2)$ and $([s_1, s_2], c_3)$. The accepted endpoint remains eligible in both rules. Although Strict's pairwise predicate is contained in Progress's, their independently scanned edge sets need not be nested.

Chronological indices strictly increase, so construction terminates without cycles. With fixed-key evidence, retained edges also decrease the appropriate verdict/test severity rank. These are builder invariants, not evidence of a learned ZPD effect. Binary accepted/non-accepted mining retains 14,027 of 16,973 Strict pairs and 18,009 of 21,416 Progress pairs. Models retrained on them agree with the canonical ones on 84.6% and 87.5% of Seen repair decisions (Section IV-E), so we read the ranking as operational rather than universal.

D. Train

The goal of Train is to learn each relation independently while keeping model and optimization comparable. The implementation fine-tunes Qwen2.5-Coder-7B-Instruct with 4-bit QLoRA [22], [23]. Adapters have rank 16, scaling 32, and dropout 0.05, targeting attention and feed-forward projections. Canonical training uses one epoch, learning rate 2×10^{-4}, and seeds 2027–2029. Assistant target tokens supply cross-entropy loss. The role-neutral prompt contains the statement, limits, permitted submissions, execution/edit summaries, and a request for a complete solution. It names neither the rule nor a preferred patch size, so specialization must come from the data rather than explicit policy instructions.

Targets are future submissions available offline; inference inputs contain only observed states. Current-only controls reset positions and remove earlier entries, preventing length leakage through position labels. Answer training is already single-source, so supplying history at inference creates a distribution change. The history-removal retraining control instead holds targets fixed and changes earlier-submission serialization. Source-matched target training holds current input and count fixed. These designs answer different questions and are not pooled into one claim about history.

The paired-target control inner-joins records by problem, user, submission identifier, and exact current code, retaining sources with distinct Answer and Progress targets. Its 7,389 training and 931 validation examples per condition use identical current-only inputs. The separate peer control requires another user on the same problem and in the same split, with exactly equal target execution evidence. For deterministic semantic-token diff distance d, the discrepancy $\delta = |d(c, y_v) - d(c, y_u)|$ must satisfy $\delta \leq 2$ or $\delta / \max(1, d(c, y_u)) \leq 0.1$. Scarce evidence groups are visited first; ties prioritize distance discrepancy, reuse, and identifier, with donor reuse capped at three. This greedy rule yields 5,231 training and 306 validation examples per condition. Equal outcomes and nearby distances do not equate structure or style, so its measured effect belongs to the entire replacement rule, not isolated provenance.

E. Select

The goal of Select is to deploy a bounded portfolio chosen without final-test outcomes. For validation inputs U, let R_v^B be inputs repaired by checkpoint v within edit budget B. Selection maximizes:

$$f_B(S) = \frac{1}{|U|} \left| \bigcup_{v \in S} R_v^B \right|, \qquad |S| = 3. \tag{2}$$

Intuitively, Equation (2) rewards complementary successes, not low training loss or the presence of every relation. Validation uses one fixed-hash trajectory from each of 461 eligible problems. The mixed pool has nine checkpoints and 84 triples; requiring one checkpoint per relation instead permits 27. The primary unconstrained mixed choice is Answer2027, Answer2028, and Progress2027. The relation-constrained choice is a different condition. Answer-9Choose3 receives nine terminal checkpoints and the same 84-triple search. Ties use recorded pass-rate, improvement, and deterministic identifier priorities. Budget-indexed selection repeats the declared objective at $B \in \{5, 10, 20, 40, 80, 160\}$, without choosing a budget from test results.

Selected checkpoints and order are frozen. Each emits at most one independent candidate, and the first observed acceptance stops generation. If none passes all tests, Algorithm 1 prioritizes pass fraction and then smaller distance. Because unchanged code is feasible, for the same fixed execution measurements the returned pass fraction cannot decrease, and a finite edit budget cannot be exceeded. This is not per-test non-regression: equal fractions can conceal gains and losses on different tests. It also does not guarantee non-regression under fresh execution, an unseen suite, or resource variability. Development explicitly measures these stronger failure modes.

For fixed independent candidates, order changes calls and patch identity but not the existence of an eligible acceptance. Stored candidates permit exact selection replay, not a guarantee of generation quality. Terminal pairing is linear in trajectory length; replaying n submissions on m tests requires $O(nm)$ executions and evidence storage. The 84-triple search is bounded at our pool size, not a scalable optimizer for arbitrary pools.

IV. Experiments

We evaluate the retained implementation and completed development evidence through five research questions:

- **RQ1: Repair effectiveness.** Which configuration provides the strongest unrestricted repair performance?
- **RQ2: Target construction and locality.** How do matched targets affect correctness and source preservation?
- **RQ3: Component efficacy.** What do candidate breadth, checkpoint identity, and serialized history contribute?

TABLE II
CORPUS AFTER WHOLE-TRAJECTORY FILTERING. PREFIXES PRECEDE RELATION FILTERING. SEEN PROBLEMS/TESTS ARE SHARED ACROSS ITS PARTITIONS; USER COUNTS ARE NOT ADDITIVE.

Group	Partition	Problems	Tests	Traj.	Submissions	Prefixes	Users
Seen	Train	492	9,446	14,027	54,481	40,454	4,012
	Valid	492	9,446	1,754	6,719	4,965	1,248
	Test	492	9,446	1,754	6,598	4,844	1,250
Unseen	Test	54	642	259	954	695	236
Total		546	10,088	17,794	68,752	50,958	–

- **RQ4: Generality.** Which findings replicate across problems, prefixes, scales, and Java populations?
- **RQ5: Development diagnosis.** Do completed history-specific modifications satisfy their fixed adoption criteria?

A. Experimental Settings

Benchmarks. Table II describes the Project CodeNet Python800 corpus [24]. We retain trajectories with at least three submissions, a non-AC state before the first acceptance, and an accepted endpoint in the benchmark reference set. Users must appear in at least three problems, and retained problems contain at least two trajectories. The refined corpus has 546 problems, 21,454 trajectories, 99,432 submissions, and 10,088 tests. Whole-trajectory 4,096-token filtering removes 3,660 trajectories before splitting. Problems ranked by eligible trajectory count supply the top 90% to Seen (492) and the remaining 54 low-resource problems to Unseen. Seen trajectories are split 80:10:10 within every problem, retaining at least one in each partition. Users can overlap across Seen partitions; "–" avoids counting repeated users as distinct people in the total row.

The primary endpoint is the last recorded non-AC submission of an eventually successful trajectory. Runtime eligibility requires the current program to remain non-AC and the recorded accepted endpoint to pass the available tests on replay. This leaves 997 of 1,754 Seen test trajectories, from 328 problems, and 250 of 259 Unseen trajectories, from all 54 problems. Excluded trajectories do not enter the denominator. CodeNet users are online-judge participants, not a verified classroom population. We also use TIKTOC's release of authentic CS1 Java CodeWorkout submissions and tests [25]: 1,125 trajectories from 224 students and 17 exercises, with 181 test trajectories from 33 held-out students. A separate exercise split has 10/3/4 train/validation/test exercises and 605/216/304 trajectories. These populations remain separate from the Python estimates.

Baselines. Zero-shot uses the same base model without fine-tuning and appends verdict/pass-count feedback after failure. LSGen retains repair-pair filtering, edit-vector retrieval, diff-guided generation, and re-retrieval [14]; its same-problem Seen-Train references are unavailable under our Unseen information constraint, so that condition is not evaluated. Answer-1 uses one terminal checkpoint. Answer-3Seed uses seeds 2027–2029 on the same 40,454 terminal pairs, controlling independent training without heterogeneous targets. Answer-9Choose3 adds seeds 2030–2035 and selects three from nine. History and current-only prompts are explicitly labeled. Mixed-9Choose3 denotes the primary unconstrained ZPDPATCH pool. Current-only reselection from the mixed pool chooses the same three Answer checkpoints as Answer-3Seed, not a relation-mixing improvement.

Other settings. Canonical generation and training use a single NVIDIA RTX 5090. Candidates execute in network-disabled containers with Python or Java 21, a 2.5-second per-test timeout, and a 2GB memory limit. The primary comparison permits at most three generations, each capped at 4,096 output tokens; this cap differs from pre-split whole-trajectory filtering. Historical Transformers deployment and the later vLLM factorial are separate inference stacks. Decoder-matched contrasts remain within a stack. Adapters use three declared seeds, but a selected portfolio's outcome is not a mean over three independent test experiments.

Repair Rate (RR) is the fraction of eligible inputs whose returned program passes every test. Pass Rate (PR) averages passed-test fractions; Improvement Rate (IR) records an increased fraction. AST tree edit distance (TED) measures current-to-returned change [26], [27] and is undefined when either program does not parse. Such cases remain in unrestricted RR but not defined-TED summaries; finite gates reject undefined-distance changes. Token and nonempty-line retention measure the original code retained, not learning value. Locality contrasts use jointly repaired instances to avoid unrelated success sets, but remain conditional on joint success.

Paired intervals resample problems 10,000 times with recorded seeds; Java reports student- and exercise-cluster alternatives [28]. We distinguish instance-weighted RR from equal-problem-weight contrasts and report Wilson intervals, exact McNemar tests, and Holm correction where specified [29]–[31]. Protocols and amendments identify when each control was frozen; the extended study was not externally preregistered. Repeatedly inspected development results are exploratory even when an individual run uses fixed gates. We use the preserved whitespace-sensitive corrections rather than mixing original and corrected counts.

Research process. OpenAI Codex assisted with design, dataset and analysis code, remote orchestration, diagnostics, and manuscript preparation under author instructions. Automated agents executed authorized jobs and inspected saved outputs; we do not claim authors manually ran every job.

Outcomes come from retained execution records and analyses, not generated prose. The experiment program stopped at the declared cutoff of 21 October 2026, 13:00 KST. This synthesis adds no training, generation, testcase rerun, or model promotion.

B. RQ1: Repair Effectiveness

TABLE III
PRIMARY PYTHON EVALUATION. RR IS SUCCESSFUL RETURNS DIVIDED BY 997 SEEN OR 250 UNSEEN INPUTS; TED AVERAGES DEFINED DISTANCES. ALL PORTFOLIOS PERMIT AT MOST THREE CALLS; ANSWER-1 PERMITS ONE. H DENOTES HISTORY INPUT, C CURRENT-ONLY INPUT. NO CROSS-SPLIT RANKING IS IMPLIED.

Method	Input	Seen		Unseen	
		RR (%)	TED	RR (%)	TED
Zero-shot	H	30.7	27.25	62.0	17.46
LSGen	H	80.2	88.27	–	–
Answer-1	C	58.1	30.27	69.6	15.30
Answer-3Seed	H	61.7	24.52	73.2	13.38
Answer-9Choose3	H	59.8	25.46	74.8	15.19
Mixed-9Choose3	H	61.5	21.05	74.8	11.40
Answer-3Seed	C	72.0	37.94	81.2	16.01
Answer-9Choose3	C	69.5	35.41	82.0	20.06

Results. Table III identifies current-only Answer-3Seed as the retained deployment configuration within our implementation, at 72.0% Seen and 81.2% Unseen. It repairs 718/997 and 203/250 inputs. Relative to current-only Answer-1, its paired gain is 13.9 points [11.71, 16.29] and 11.6 points [7.32, 16.23]. This contrast changes both checkpoint identity and opportunity count; RQ3 separates them. It is not universally best: current-only Answer-9Choose3 repairs two more Unseen inputs (205/250), but 25 fewer Seen inputs (693/997). We retain the simpler fixed triple rather than selecting a different final method for each observed test split. Current-only mixed-pool reselection chooses exactly that terminal triple, providing no evidence that relation diversity improves this deployment setting.

The history-input mixed portfolio repairs 613/997 Seen and 187/250 Unseen inputs. Its Seen gain over Zero-shot is 30.8 points [26.43, 35.09], or 26.1 [21.24, 30.93] with equal problem weights. Mixed-only and Zero-shot-only successes number 342 and 35 (McNemar $p < 10^{-63}$). This is not a history-specific effect: Mixed nearly ties Answer-3Seed, while its matched nine-checkpoint contrast is +1.7 points [-0.79, 4.29] on Seen and 0.0 [-3.70, 3.75] on Unseen.

LSGen repairs 800/997 Seen inputs versus Mixed's 613/997, a difference of -18.8 points [-23.04, -14.46]; 80 versus 267 exclusive successes give $p < 10^{-23}$. On joint repairs, Mixed's TED is lower by 43.34 [38.47, 48.44] relative to LSGen and 8.46 [5.05, 12.55] relative to Zero-shot. Conditioning excludes unilateral successes, so smaller patches do not cancel the primary correctness deficit. LSGen's problem-held-out condition remains unevaluated.

Answer to RQ1. We retain current-only Answer-3Seed within our implementation. It is neither an overall winner nor a new ZPD success: LSGen leads on Seen, current-only Answer-9Choose3 has slightly higher Unseen RR, and mixed targets lack an established unrestricted advantage. Edit-constrained repair is a separate objective.

TABLE IV
SOURCE- AND COUNT-MATCHED TARGET REPLACEMENT. RR USES ALL ELIGIBLE INPUTS; RETENTION DIFFERENCES ARE PROGRESS MINUS ANSWER ON JOINT REPAIRS. JAVA USES HELD-OUT STUDENTS. POSITIVE RETENTION DIFFERENCES FAVOR SOURCE PRESERVATION, NOT DEMONSTRATED LEARNING.

Population	RR (%)		Retention (pp)	
	Answer	Progress	Tokens	Lines
Seen (997)	53.2	34.0	+5.6	+8.9
Unseen (250)	76.8	65.6	+6.5	+10.4
Java (181)	85.6	50.8	+4.8	+5.1

C. RQ2: Target Construction and Locality

Matched targets. Table IV changes the target while holding current inputs and training counts fixed. Progress-minus-Answer RR is -19.2 points [-22.96, -15.54] on Seen and -11.2 [-17.37, -4.88] on Unseen, with exact McNemar $p = 7.85\times10^{-34}$ and $p = 6.17\times10^{-5}$. Thus the canonical terminal dataset's advantage cannot be attributed only to having more training records. The paired population is restricted to sources with distinct targets, however, and does not estimate an effect for all CodeNet submissions. Moreover, a Progress target can itself be accepted. The intervention is the retained-evidence target rule, not a clean contrast between exclusively incorrect and exclusively correct targets.

Progress's locality benefit is visible on joint successes. Among 299 Seen and 154 Unseen jointly repaired inputs, 278 and 134 have defined source-to-repair TED for both methods. Answer-minus-Progress mean TED is 10.08 [7.53, 12.99] and 6.83 [4.58, 9.39]. Token retention improves by 5.6 [4.29, 6.94] and 6.5 [4.26, 8.95] points; line retention improves by 8.9 [6.64, 11.16] and 10.4 [6.95, 14.25]. The 21 and 20 undefined-TED cases remain in RR and are not declared unrepairable. These measures consistently describe more source-preserving successful outputs, but they neither establish easier comprehension nor explain which method would best help a particular learner.

Direct retraining on the student-held-out Java population reproduces this direction without relying on Python AST distance. Terminal targets repair 155 trajectories and Progress targets 92, a difference of 34.8 points; Progress-minus-Answer intervals are [-44.91, -25.47] under student clustering and [-47.31, -23.79] under exercise clustering. Among 90 joint repairs, Progress targets retain 4.8 more lexical-token points and 5.1 more line points, with intervals [2.79, 7.19] and [2.92, 7.25]. This is evidence for a coverage–locality tradeoff across these two languages and populations. It is not evidence that the same magnitude generalizes to other classrooms, nor a replication of the Python peer-target intervention.

Peer targets. The separate outcome- and distance-matched replacement raises RR from 50.4% to 54.7% on Seen and

from 78.4% to 84.4% on Unseen. Same-minus-peer intervals are [-7.71, -0.84] and [-11.79, -0.41], with exact McNemar $p = 0.005$ and $p = 0.032$. Only 24.4% of canonical training records and 11.4% of validation records admit this match. The result therefore concerns the specified caliper, donor cap, source population, and replacement rule. It does not prove that user identity alone causes the difference, since equal outcomes and nearby edit distance still permit differences in target structure, style, and length. It also cannot support a claim that one's own trajectory is uniquely necessary: matched peer targets improve coverage in this control.

TABLE V
SEEN REPAIR RATE UNDER FIXED TED BUDGETS. EACH ROW HAS DENOMINATOR 997. MIXED IS THE BUDGET-SPECIFIC VALIDATION SELECTION, NOT AN ORACLE CHOICE PER TEST INPUT. LSGEN PRESERVES ITS THREE-GENERATION CANDIDATE BUDGET. UNRESTRICTED RESULTS REMAIN IN TABLE III.

TED budget	Answer-3Seed, C	Mixed, H	LSGen
5	16.9	22.5	4.8
10	28.5	32.4	8.0
20	40.3	44.0	13.9
40	51.4	50.9	30.4
80	61.0	56.0	50.8
160	66.8	57.9	67.6

Operational locality. Table V gives patch size an explicit engineering meaning: the user permits only changes within a declared structural budget. At TED 20, Mixed repairs 44.0% of Seen inputs versus LSGen's 13.9%, a 30.1-point difference [26.45, 33.82]. Across the six prespecified budgets its mean advantage over LSGen is 14.7 points [12.03, 17.43]. This is a budget-indexed outcome, not an unrestricted superiority claim: LSGen wins at 160, and current-only Answer-3Seed overtakes Mixed from 40 onward. On Unseen, Mixed versus current-only Answer-3Seed is 38.8% versus 32.0% at 5, 48.0% versus 40.8% at 10, and 57.6% versus 54.8% at 20; the order reverses at 40. No observed budget is retroactively designated the unique primary endpoint.

A typical case selected near the median distance gap among 341 compact joint repairs concerns a counter reset while detecting consecutive doubles. Progress changes the reset condition; Answer inserts an internal guard. Both pass and have TED 7. The mean locality effect need not characterize every pair, and neither repair demonstrates a learner's next independent action.

Answer to RQ2. Target construction changes both coverage and successful-patch locality under matched inputs, with the same direction on student-held-out Java. Progress targets are more source-preserving but materially less successful. Small-budget repair is an operational use case for that tradeoff; pedagogical benefit remains unmeasured. The peer result also prevents interpreting the effect as uniquely personal supervision.

TABLE VI
CURRENT-ONLY FACTORIAL WITHIN THE LATER VLLM STACK. EACH CELL USES THE SAME ELIGIBLE 997/250 INPUTS. THESE ABSOLUTE RATES MUST NOT BE SUBTRACTED FROM THE HISTORICAL-STACK RATES IN TABLE III TO ESTIMATE CHECKPOINT EFFECTS.

Generation condition	Seen (%)	Unseen (%)
One greedy checkpoint	53.1	67.2
Three greedy checkpoints	62.9	78.0
Same-checkpoint stochastic triple	66.8	79.6
Distinct-checkpoint stochastic triple	68.9	80.8

D. RQ3: Component Efficacy

Fixed candidate breadth. For three stored draws with success indicators a_{ij}, the expected uniformly selected one-draw success for input i is $\frac{1}{3}\sum_j a_{ij}$, whereas the three-draw union succeeds when $\max_j a_{ij} = 1$. Their paired difference isolates additional opportunities within the identical candidate set. Replacing greedy decoding by one stochastic draw costs 7.6 points [-9.81, -5.41], whereas execution-selecting three of those identical fixed draws contributes 15.1 points [13.48, 16.76] on Seen. This is a descriptive decomposition of stored executions, not a claim that three future draws have guaranteed success or independent errors. It directly demonstrates why comparing a portfolio only with one adapter overstates evidence for its specialized training signal.

The temperature sweep retains all declared conditions instead of selecting the best one for the headline. Through $T = 1.0$ the one-draw rate stays within 41.1–43.1% on Seen, the three-draw union reaches 60.0/75.2%, and at $T = 1.5$ the decoder collapses to 15.8/35.2%. A separate 20-draw stream at $T = 0.8$ reaches 74.8% Seen and 84.0% Unseen, using on average 7.44 and 5.08 calls under early stopping. Those higher rates spend a different generation budget and do not replace the three-call primary results. Fine-tuning also matters: the matched base-model three-draw union reaches only 30.4/65.6%, so adaptation adds 27.2 points [22.99, 31.41] on Seen and 6.4 points [1.19, 11.34] on Unseen. Breadth alone therefore does not explain all improvement over the base model.

At matched temperature, replacing three draws of one checkpoint by one draw from each of three checkpoints changes RR by -0.1 points [-2.00, 1.80] on Seen and 2.4 points [-0.41, 5.31] on Unseen. These intervals do not establish a consistent advantage from checkpoint diversity. They also do not show equivalence: the uncertainty still admits benefits and harms of practical interest. A deterministic multi-seed gain can coexist with this result because sampling from one checkpoint already supplies diversity. The decoder must be controlled before an ensemble result is attributed to independently learned policies.

The current-only factorial in Table VI tests this explanation on one inference stack. Three greedy checkpoints gain 9.8 [7.85, 11.92] and 10.8 [7.23, 14.64] points over one, but under the matched stochastic decoder distinct checkpoints add only 2.1 points [0.00, 4.16] and 1.2 points [-2.93, 5.93]. The equal-

problem Seen interval includes zero, so the rounded instance-weighted lower bound does not establish a robust checkpoint benefit. Absolute greedy-three coverage is 62.9/78.0% on that stack against 72.0/81.2% on the historical one. Backend sensitivity is material, and the deployment recommendation is restricted to the evaluated stack rather than asserted as backend-independent superiority.

History and portfolio selection. Retraining with the same targets but without earlier-submission context yields no significant RR difference in the four principal tests after Holm correction; the smallest adjusted p is 0.313. This negative result does not localize gains to trajectory-derived relations. At the portfolio level, removing history improves frozen mixed and terminal RR by 8.8/11.1 Seen points and 7.6/6.0 Unseen points, with all intervals excluding zero. The result is compatible with a prompt-distribution cost, especially because terminal training uses single-source inputs. It is inconsistent with describing history as necessary to the measured portfolio advantage. The matched retraining and frozen-input interventions should not be treated as interchangeable estimates.

Selection-space controls also fail to establish a general heterogeneous-target benefit. Answer-9Choose3 receives the same nine-checkpoint pool size and 84-subset opportunity as Mixed. The primary five-fold cross-fitted mixed-minus-terminal contrast is +2.2 points [-0.19, 4.63], compared with the non-cross-fitted +1.7 [-0.79, 4.29]. These estimates are positive but uncertain. Exact replay of the six orders of a fixed triple checks the independent-candidate controller, not the value of learned diversity. There is consequently no basis for turning a verified ordering property or portfolio certificate into a theoretical guarantee about learned repair quality.

Answer to RQ3. Candidate breadth and adaptation each contribute measured gains. Distinct checkpoints add no consistent advantage once the stochastic decoder is matched, and serialized history is not supported as a necessary input. Target heterogeneity has a small, uncertain unrestricted contrast against the equally searched terminal pool. These controls constrain the mechanism claim rather than merely supplying additional baselines.

E. RQ4: Generality

Problems and execution suites. Within Unseen, Mixed exceeds Zero-shot by 12.8 points [6.83, 18.75], with 45 versus 13 exclusive successes and exact McNemar $p = 3.01 \times 10^{-5}$. Among 122 joint repairs it reduces TED by 6.41 [3.43, 9.89]. The higher absolute Unseen than Seen RR is not evidence of stronger generalization: the partitions differ in trajectory availability and problem composition. Matching all 54 Unseen problems to distinct Seen problems on six pre-repair difficulty covariates changes the raw gap into -4.6 equal-problem points [-14.18, 5.30]. We therefore rely on within-split paired contrasts, not the direction of the raw cross-split difference. Matching reduces measured imbalance but cannot balance unobserved difficulty.

A complementary observed/hidden-slice check partitions the existing authored suite. On Unseen, Mixed and Answer-9Choose3 jointly pass observed and held-back slices on 189/250 and 184/250 inputs, respectively. Conditional confirmation among observed-suite successes is 189/192 (98.4%) and 184/186 (98.9%); 13 versus 8 exclusive joint successes yield $p = 0.383$. Repeating the procedure on all 328 Seen problems with a distinct seed yields 97.8% and 97.9% confirmation, but joint success is 452/997 versus 469/997. High confirmation thus does not demonstrate that Mixed wins. These are held-back slices of the same authored tests, not independently authored specifications, and the large conditional percentage must not replace the full-denominator repair rate.

Earlier prefixes. Across 2,770 Seen and 674 Unseen executable prefixes of eventually successful trajectories, current-only Answer-3Seed reaches 69.6% and 80.1% RR. The early, middle, and last thirds reach 70.0%, 67.6%, and 71.4% on Seen and 82.9%, 77.3%, and 81.8% on Unseen. This extends evaluation beyond the last failed submission and finds no simple monotonic position effect. It does not cover trajectories that never reach acceptance, and multiple prefixes from one trajectory are dependent. The 2,770-prefix rate is neither added to the 997-input result nor reported as an independent replication on new users.

Model scale and classroom data. At 1.5B parameters three checkpoints still gain 13.4 points over one [11.16, 15.92], but mixed-minus-terminal is -0.8 [-2.62, 1.07]. Mixed reaches 50.25% and the nine-checkpoint terminal control 51.05% on Seen. This smaller model reproduces the utility of multiple opportunities, not an unrestricted mixed-target advantage. It is one scale change within a model family, not broad model-family coverage. No proprietary frontier model comparison was completed.

The Java portfolio results differ from RQ2's direct matched-target retraining. Mixed-minus-Answer-9Choose3 is positive on four held-out exercises (3.9 points [2.99, 5.11]) and inconclusive on held-out students (0.6 [-1.20, 2.59]). Absolute RR is 56.25% versus 52.30% over 304 exercise-held-out trajectories and 83.98% versus 83.43% over 181 student-held-out trajectories. The exercise estimate rests on four clusters only: refitting with one exercise left out keeps the contrast between 3.6 and 4.4 points, and single exercises range from 2.5 to 5.5 points. This sensitivity check preserves the sign on those exercises, but four clusters cannot support a broad classroom-population claim. The matched-target tradeoff has clearer cross-population support than a universal portfolio benefit.

Representation and overlap. Binary accepted/non-accepted mining removes the fine severity ordering but also changes the target set. Retraining changes Progress RR by +2.3 Seen and -2.8 Unseen points in the canonical-minus-alternative direction. The nonuniform changes show that the ordering is not an innocuous implementation detail. Similarly, train/test overlap deserves measurement rather than an assertion of absence. For Mixed and Answer-9Choose3, exact matches between a returned repair and a training-time accepted program occur in 0.6% and 1.3% of selections, near-duplicates above a 0.90 token-similarity threshold in 3.8% and 5.3%, and median max-

TABLE VII
CHRONOLOGICAL TWO-SLOT DEVELOPMENT BLOCKS, EACH WITH 320 INPUTS. CELLS ARE STABLE ACCEPTED / REGRESSING / UNKNOWN COUNTS (NOT DISJOINT). FOLDS COUNT H ACCEPTANCE NON-WORSE THAN C. EVERY ROW FAILS THE FIXED ADOPTION RULE. DIFFERENT MODELS, CANDIDATES, AND EXECUTION BLOCKS PROHIBIT A POOLED RANKING.

Modification	C	H	HS	U	Folds
Current code last	208/10/16	144/16/16	145/17/16	148/19/17	0/5
Full historical source	202/9/17	169/13/16	165/12/17	190/15/16	0/5
Matched C/H fine-tuning	209/10/10	192/15/11	198/13/11	206/13/14	0/5
Shared current candidate	200/15/22	194/19/19	198/17/19	199/18/20	2/5
Public tie-breaking	192/14/21	195/13/20	198/12/20	202/12/19	3/5
Edit-balanced fine-tuning	195/15/15	200/17/16	197/17/16	193/25/17	3/5
Line-span splice output	108/14/16	101/16/17	98/16/17	103/13/16	2/5
Content-anchor output	136/18/19	138/26/19	138/24/20	136/16/18	3/5
Public-feedback revision	194/12/14	193/10/14	192/13/14	202/14/14	2/5
Counterexample revision	197/12/12	188/12/12	191/12/11	202/15/11	1/5
De-anchored revision	204/13/13	190/12/14	194/13/13	201/14/15	1/5
Likelihood selection	173/3/13	161/5/12	161/5/12	166/9/12	1/5

imum similarity to a same-problem training target is 66.7% and 68.6%. Low exact-copy rates do not exclude structural reuse, pretraining contamination, or user-level overlap [32].

Answer to RQ4. The matched-target tradeoff replicates on authentic student-held-out Java, and candidate breadth remains useful at smaller model scale. Mixed-target unrestricted superiority does not replicate consistently. Prefix, difficulty, overlap, verdict-order, and hidden-slice checks narrow specific threats without converting CodeNet into a classroom study or test-suite acceptance into semantic correctness.

F. RQ5: Development Diagnosis

Decision rule. Subsequent development compared current-only (C), authentic-history (H), shuffled-history (HS), and matched other-user (U) conditions. Adoption required strictly greater pooled stable acceptance for H than every control; no increase versus C in regressions, regression upper bounds, or unknowns; all required problem folds non-worse; and completed evaluation. Regression means losing a previously passed testcase; unknown means unresolved execution. These requirements exceed the fixed-measurement pass-fraction invariant of Algorithm 1. Historical verdicts also differ from replay: only 7,979 of 14,027 unique training targets were cached as accepted. We retain runtime uncertainty rather than interpreting it as learner ability. Detailed audits and predictive studies remain in the artifact; predictive loss is not RR.

History and generation. Table VII preserves all twelve main two-slot modifications. Prompt reordering, full-source histories, and matched fine-tuning fail to distinguish authentic history. A crossed diagnosis gives the H adapter 201 acceptances on current code, 192 on H, 198 on HS, and 206 on U: longer-input adaptation is not an authentic-content benefit. Edit-balanced training produces the highest observed two-slot H rate, 200/320 (62.5%), but increases regressions (17 versus 15), unknowns (16 versus 15), and regression upper bounds (33 versus 30), with only 3/5 required non-worse folds. H selection misses just seven known accepted candidates; 97 inputs lack a confirmed accepted candidate and 16 remain unresolved. Selection alone cannot eliminate this generation deficit.

Splice output causes 48 format failures, seven resource failures, and 151 valid-to-invalid syntax changes across 640 H attempts. Content anchors reduce representation failures but still yield only 138 acceptances and 26 regressions. Public-feedback and counterexample revision do not pass controls. Hiding failed generated code reduces copying and improves some public-test outcomes, yet final H acceptance remains 190 versus C 204. Likelihood selection then reduces H to 161, with 10 gains and 39 losses; 34 losses select unchanged code. These diagnostics distinguish invalid edits, insufficient candidates, public-test overfitting, and excessive preservation rather than treating every failure as a ranking problem.

Separate eight-draw bank. An earlier 320-input, 23-problem bank includes 19 initially accepted programs, 288 non-accepted programs, and 13 unknowns; accepted/320 is not failed-input RR. Structural and history-feature refinements reach 203–211 acceptances but fail their original regression and 4/5-fold requirements or match perturbation controls. Conservative preservation reduces regressions but returns unchanged code 155 times, leaving 122 acceptances. Generic public-execution selection is a genuine improvement: C reaches 252/320 (78.75%), with nine regressions and 13 unknowns, versus the prior selector's 211/320, 51, and 13. It passes its separate 22-check execution-component gate and all five folds. However, H reaches 251/320 with identical regression/unknown counts and does not separate from shuffled controls. Added execution cost and unguaranteed suite independence further prevent ranking this bank against the primary 997/250-input study.

Answer to RQ5. No completed candidate passes all original history-specific criteria. Earlier pilots and fresh 128-input checks also failed; detailed negative results remain in the final artifact report. The last matched preference run completed 19,199 training and 4,365 validation examples per C/H arm but had no repair inference or testcase evaluation by the cutoff: it has no RR and is not eligible for promotion. We retain the eval-

uated current-only terminal triple. These adaptively inspected runs diagnose failure modes, not independent confirmatory or educational benefits.

G. Threats to Validity

Internal validity. Runtime variability, undefined source ASTs, and Python whitespace affect labels and distances. We retain unknowns, denominators, and whitespace-sensitive corrections; hashes establish provenance, not correctness. Source and peer matching do not equate all target semantics or styles. Prompt-distribution compatibility and inference stacks also affect outcomes, requiring separate within-stack, frozen-input, and matched-retraining contrasts.

Construct validity. Acceptance is not proof of semantic correctness [33]; same-suite slices are not independent specifications. TED and retention do not measure cognitive load or subsequent learning. CodeNet does not identify novices or measure mastery. Java supplies classroom submissions, not an intervention observing feedback use. Historical verdicts and fresh execution also differ. These distinctions limit educational and target-quality interpretations.

External and conclusion validity. Filtering favors short, eventually successful Python trajectories and permits Seen user overlap. Java adds one course context and only four held-out exercises. Two scales of one model family do not establish broad generality. Cluster bootstrap and equal-problem estimates do not eliminate small-cluster uncertainty, multiplicity, or adaptive-development bias. Gates were frozen before individual runs, not the entire program; reused development results remain exploratory.

V. Related Work

Repair and generation budgets. Generate-and-validate repair spans search, learned transformations, and language models [1]–[4]. Search analyses [34] and execution-based evaluation [35], [36] distinguish candidate quality from exposure. Neural patches, ensembles, synthetic corruption, and execution-informed self-supervision provide different training signals [19], [20], [37], [38]; zero-shot studies examine repair without task adaptation [39], [40]. Independent ensembles and weight averaging supply other notions of diversity [41], [42]. Our adapters execute separately; we do not introduce portfolio repair. We isolate trajectory targets with same-target seeds, equal search spaces, fixed draws, and current-only controls.

Programming-exercise feedback. Educational repair exploits correct programs through clustering, alignment, and refactoring [8]–[10]; related representations and tutoring systems propagate feedback and generate hints [43]–[46]. Language models support repair, syntax feedback, conversations, peers, and counterexamples [6], [7], [11]–[13], [47], [48]. Assignment-aware and diversified feedback use further reference strategies [49]–[51]. LSGen's iterative edit-driven retrieval [14] retains its strong Seen advantage here. Benchmarking and surveys emphasize population and protocol differences [5], [21], [52]. These methods are not uniformly terminal-target learners. Our narrower intervention replaces the target for matched sources, exposing a coverage–locality tradeoff rather than dominance over all feedback strategies.

History and learner evidence. ZPD and scaffolding concern supported performance, not program edit distance [15]–[18]. Knowledge tracing models proficiency from interactions [53]–[55]; TIKTOC incorporates testcase evidence [25]. Next-step hints, feedback ladders, and novice-response studies examine guidance more directly [56]–[58], while feedback theory emphasizes its interpretation and use [59]–[62]. We test the narrower repair prerequisite: whether authentic history improves generated changes beyond current code and perturbed histories. The results do not establish that prerequisite. Educational claims additionally require comprehension, transfer, and independent-correction measurements, not only executable outputs.

VI. Conclusion

We presented ZPDPatch, a Mine–Train–Select technique for constructing trajectory-derived repair targets and evaluating execution-guided portfolios. Matched Python and student-held-out Java experiments show a reproducible coverage–locality tradeoff: progress targets preserve more source code on joint successes but repair fewer inputs than terminal targets. Candidate breadth and adaptation explain substantial gains, whereas authentic-history conditioning and target heterogeneity do not establish consistent unrestricted advantages. Completed development improves generic execution selection but supplies no candidate passing all history-specific adoption conditions, so we retain the existing current-only terminal triple rather than promote an unverified revision. Educational benefits and broader model-family generality remain questions for future studies.

Data Availability

The retained artifact includes code, split manifests, release hashes, outcomes, protocols, negative results, and a claim–evidence ledger. Upstream data are CodeNet and TIKTOC/CodeWorkout [24], [25]; redistribution must respect their terms. An anonymous deposit is pending, so public artifact access is not claimed. We report aggregate secondary-data results, a paraphrased example, and no learner intervention. Automated assistance is disclosed in the settings.

REFERENCES

[1] L. Gazzola, D. Micucci, and L. Mariani, "Automatic software repair: A survey," *IEEE Transactions on Software Engineering*, vol. 45, no. 1, pp. 34–67, 2019.

[2] M. Monperrus, "Automatic software repair: A bibliography," *ACM Computing Surveys*, vol. 51, no. 1, pp. 1–24, 2018.

[3] W. Weimer, T. Nguyen, C. Le Goues, and S. Forrest, "Automatically finding patches using genetic programming," in *Proceedings of the 31st International Conference on Software Engineering*, 2009, pp. 364–374.

[4] C. Le Goues, T. Nguyen, S. Forrest, and W. Weimer, "Genprog: A generic method for automatic software repair," *IEEE Transactions on Software Engineering*, vol. 38, no. 1, pp. 54–72, 2012.

[5] H. Keuning, J. Jeuring, and B. Heeren, "A systematic literature review of automated feedback generation for programming exercises," *ACM Transactions on Computing Education (TOCE)*, vol. 19, no. 1, pp. 1–43, 2018.

[6] H. Joshi, J. C. Sanchez, S. Gulwani, V. Le, G. Verbruggen, and I. Radiček, "Repair is nearly generation: Multilingual program repair with llms," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 4, 2023, pp. 5131–5140.

[7] J. Zhang, J. P. Cambronero, S. Gulwani, V. Le, R. Piskac, G. Soares, and G. Verbruggen, "Pydex: Repairing bugs in introductory python assignments using llms," *Proceedings of the ACM on Programming Languages*, vol. 8, no. OOPSLA1, pp. 1100–1124, 2024.

[8] S. Gulwani, I. Radiček, and F. Zuleger, "Automated clustering and program repair for introductory programming assignments," *ACM SIGPLAN Notices*, vol. 53, no. 4, pp. 465–480, 2018.

[9] K. Wang, R. Singh, and Z. Su, "Search, align, and repair: data-driven feedback generation for introductory programming exercises," in *Proceedings of the 39th ACM SIGPLAN conference on programming language design and implementation*, 2018, pp. 481–495.

[10] Y. Hu, U. Z. Ahmed, S. Mechtaev, B. Leong, and A. Roychoudhury, "Refactoring based program repair applied to programming assignments," in *Proceedings of the 34th IEEE/ACM International Conference on Automated Software Engineering (ASE)*. IEEE, 2019, pp. 388–398.

[11] F. Liu, Z. Liu, Q. Zhao, J. Jiang, L. Zhang, Z. Sun, G. Li, Z. Li, and Y. Ma, "Fastfixer: An efficient and effective approach for repairing programming assignments," in *Proceedings of the 39th IEEE/ACM International Conference on Automated Software Engineering*, 2024, pp. 669–680.

[12] B. Yang, H. Tian, W. Pian, H. Yu, H. Wang, J. Klein, T. F. Bissyandé, and S. Jin, "Cref: An llm-based conversational software repair framework for programming tutors," in *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2024, pp. 882–894.

[13] P. Orvalho, M. Janota, and V. M. Manquinho, "Counterexample guided program repair using zero-shot learning and maxsat-based fault localization," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 39, no. 1, 2025, pp. 649–657.

[14] Z. Dai, Z. Zhao, H. Wang, X. Tang, S. Wu, C. Yao, Z. Gao, and J. Chen, "Learner-tailored program repair: A solution generator with iterative edit-driven retrieval enhancement," *arXiv preprint arXiv:2601.08545*, 2026.

[15] L. S. Vygotsky, *Mind in Society: The Development of Higher Psychological Processes*, M. Cole, V. John-Steiner, S. Scribner, and E. Souberman, Eds. Cambridge, MA: Harvard University Press, 1978.

[16] T. Murray and I. Arroyo, "Toward measuring and maintaining the zone of proximal development in adaptive instructional systems," in *International conference on intelligent tutoring systems*. Springer, 2002, pp. 749–758.

[17] I.-A. Chounta, P. Albacete, P. Jordan, S. Katz, and B. M. McLaren, "The "grey area": A computational approach to model the zone of proximal development," in *European conference on technology enhanced learning*. Springer, 2017, pp. 3–16.

[18] N. Schulze Bernd and I.-A. Chounta, "Beyond the grey area: exploring the effectiveness of scaffolding as a learning measure," in *International Conference on Artificial Intelligence in Education*. Springer, 2024, pp. 365–378.

[19] M. Tufano, C. Watson, G. Bavota, M. Di Penta, M. White, and D. Poshyvanyk, "An empirical study on learning bug-fixing patches in the wild via neural machine translation," *ACM Transactions on Software Engineering and Methodology*, vol. 28, no. 4, pp. 1–29, 2019.

[20] T. Lutellier, H. V. Pham, L. Pang, Y. Li, M. Wei, and L. Tan, "Coconut: Combining context-aware neural translation models using ensemble for program repair," in *Proceedings of the 29th ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2020, pp. 101–114.

[21] C. Koutcheme, N. Dainese, S. Sarsa, J. Leinonen, A. Hellas, and P. Denny, "Benchmarking educational program repair," *arXiv preprint arXiv:2405.05347*, 2024.

[22] B. Hui, J. Yang, Z. Cui, J. Yang, D. Liu, L. Zhang, T. Liu, J. Zhang, B. Yu, K. Lu *et al.*, "Qwen2.5-Coder technical report," *arXiv preprint arXiv:2409.12186*, 2024.

[23] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "Qlora: Efficient finetuning of quantized llms," *Advances in neural information processing systems*, vol. 36, pp. 10 088–10 115, 2023.

[24] R. Puri, D. S. Kung, G. Janssen, W. Zhang, G. Domeniconi, V. Zolotov, J. Dolby, J. Chen, M. Choudhury, L. Decker *et al.*, "Project codenet: A large-scale ai for code dataset for learning a diversity of coding tasks," *arXiv preprint arXiv:2105.12655*, vol. 1035, 2021.

[25] Z. Duan, N. Fernandez, A. Hicks, and A. Lan, "Test case-informed knowledge tracing for open-ended coding tasks," in *Proceedings of the 15th International Learning Analytics and Knowledge Conference*, 2025, pp. 235–250.

[26] K. Zhang and D. Shasha, "Simple fast algorithms for the editing distance between trees and related problems," *SIAM Journal on Computing*, vol. 18, no. 6, pp. 1245–1262, 1989.

[27] M. Pawlik and N. Augsten, "Tree edit distance: Robust and memory-efficient," *Information Systems*, vol. 56, pp. 157–173, 2016.

[28] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[29] E. B. Wilson, "Probable inference, the law of succession, and statistical inference," *Journal of the American Statistical Association*, vol. 22, no. 158, pp. 209–212, 1927.

[30] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[31] S. Holm, "A simple sequentially rejective multiple test procedure," *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.

[32] M. Allamanis, "The adverse effects of code duplication in machine learning models of code," in *Proceedings of the 2019 ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software*, 2019, pp. 143–153.

[33] E. K. Smith, E. T. Barr, C. Le Goues, and Y. Brun, "Is the cure worse than the disease? overfitting in automated program repair," in *Proceedings of the 10th Joint Meeting on Foundations of Software Engineering*, 2015, pp. 532–543.

[34] Y. Qi, X. Mao, Y. Lei, Z. Dai, and C. Wang, "The strength of random search on automated program repair," in *Proceedings of the 36th International Conference on Software Engineering*, 2014, pp. 254–265.

[35] M. M. A. Haque, W. U. Ahmad, I. Lourentzou, and C. Brown, "Fixeval: Execution-based evaluation of program fixes for programming problems," in *2023 IEEE/ACM International Workshop on Automated Program Repair (APR)*. IEEE, 2023, pp. 11–18.

[36] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[37] M. Yasunaga and P. Liang, "Break-it-fix-it: Unsupervised learning for program repair," in *International conference on machine learning*. PMLR, 2021, pp. 11 941–11 952.

[38] H. Ye, M. Martinez, X. Luo, T. Zhang, and M. Monperrus, "Selfapr: Self-supervised program repair with test execution diagnostics," in *Proceedings of the 37th IEEE/ACM International Conference on Automated Software Engineering*, 2022, pp. 1–13.

[39] C. S. Xia and L. Zhang, "Less training, more repairing please: Revisiting automated program repair via zero-shot learning," in *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, 2022, pp. 959–971.

[40] N. Jiang, K. Liu, T. Lutellier, and L. Tan, "Impact of code language models on automated program repair," in *Proceedings of the 45th International Conference on Software Engineering*, 2023, pp. 1430–1442.

[41] B. Lakshminarayanan, A. Pritzel, and C. Blundell, "Simple and scalable predictive uncertainty estimation using deep ensembles," in *Advances in Neural Information Processing Systems*, vol. 30, 2017.

[42] M. Wortsman, G. Ilharco, S. Y. Gadre, R. Roelofs, R. Gontijo-Lopes, A. S. Morcos, H. Namkoong, A. Farhadi, Y. Carmon, S. Kornblith, and L. Schmidt, "Model soups: Averaging weights of multiple fine-tuned models improves accuracy without increasing inference time," in *Proceedings of the 39th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, vol. 162, 2022, pp. 23 965–23 998.

[43] C. Piech, J. Huang, A. Nguyen, M. Phulsuksombati, M. Sahami, and L. Guibas, "Learning program embeddings to propagate feedback on student code," in *Proceedings of the 32nd International Conference on Machine Learning*, 2015, pp. 1093–1102.

[44] E. L. Glassman, J. Scott, R. Singh, P. J. Guo, and R. C. Miller, "Overcode: Visualizing variation in student solutions to programming problems at scale," *ACM Transactions on Computer-Human Interaction*, vol. 22, no. 2, pp. 1–35, 2015.

[45] K. Rivers and K. R. Koedinger, "Data-driven hint generation in vast solution spaces: A self-improving python programming tutor," *International Journal of Artificial Intelligence in Education*, vol. 27, no. 1, pp. 37–64, 2017.

[46] T. W. Price, Y. Dong, and D. Lipovac, "isnap: towards intelligent tutoring in novice programming environments," in *Proceedings of the 2017 ACM SIGCSE Technical Symposium on computer science education*, 2017, pp. 483–488.

[47] T. Phung, J. Cambronero, S. Gulwani, T. Kohn, R. Majumdar, A. Singla, and G. Soares, "Generating high-precision feedback for programming syntax errors using large language models," *arXiv preprint arXiv:2302.04662*, 2023.

[48] Q. Zhao, F. Liu, L. Zhang, Y. Liu, Z. Yan, Z. Chen, Y. Zhou, J. Jiang, and G. Li, "Peer-aided repairer: Empowering large language models to repair advanced student assignments," *arXiv preprint arXiv:2404.01754*, 2024.

[49] J. Heo, H. Jeong, D. Choi, and E. Lee, "Referent: Transformer-based feedback generation using assignment information for programming course," in *2023 IEEE/ACM 45th International Conference on Software Engineering: Software Engineering Education and Training (ICSE-SEET)*. IEEE, 2023, pp. 101–106.

[50] D. Choi, J. Heo, and E. Lee, "Automated feedback generation for multiple function programs," in *2021 28th Asia-Pacific Software Engineering Conference (APSEC)*. IEEE, 2021, pp. 582–583.

[51] D. Choi and E. Lee, "Automated feedback generation for programming assignments through diversification," in *2025 IEEE/ACM 37th International Conference on Software Engineering Education and Training (CSEE&T)*. IEEE, 2025, pp. 230–241.

[52] J. Yi, U. Z. Ahmed, A. Karkare, S. H. Tan, and A. Roychoudhury, "A feasibility study of using automated program repair for introductory programming assignments," in *Proceedings of the 2017 11th joint meeting on foundations of software engineering*, 2017, pp. 740–751.

[53] A. T. Corbett and J. R. Anderson, "Knowledge tracing: Modeling the acquisition of procedural knowledge," *User modeling and user-adapted interaction*, vol. 4, no. 4, pp. 253–278, 1994.

[54] C. Piech, J. Bassen, J. Huang, S. Ganguli, M. Sahami, L. J. Guibas, and J. Sohl-Dickstein, "Deep knowledge tracing," *Advances in neural information processing systems*, vol. 28, 2015.

[55] A. Ghosh, N. Heffernan, and A. S. Lan, "Context-aware attentive knowledge tracing," in *Proceedings of the 26th ACM SIGKDD international conference on knowledge discovery & data mining*, 2020, pp. 2330–2339.

[56] L. Roest, H. Keuning, and J. Jeuring, "Next-step hint generation for introductory programming using large language models," in *Proceedings of the 26th Australasian Computing Education Conference*, 2024, pp. 144–153.

[57] H. Heickal and A. Lan, "Generating feedback-ladders for logical errors in programming using large language models," *arXiv preprint arXiv:2405.00302*, 2024.

[58] R. Xiao, X. Hou, and J. Stamper, "Exploring how multiple levels of gpt-generated programming hints support or disappoint novices," in *Extended Abstracts of the CHI Conference on Human Factors in Computing Systems*, 2024, pp. 1–10.

[59] S. Narciss, "Feedback strategies for interactive learning tasks," in *Handbook of research on educational communications and technology*. Routledge, 2008, pp. 125–143.

[60] J. Hattie and H. Timperley, "The power of feedback," *Review of Educational Research*, vol. 77, no. 1, pp. 81–112, 2007.

[61] V. J. Shute, "Focus on formative feedback," *Review of Educational Research*, vol. 78, no. 1, pp. 153–189, 2008.

[62] V. Aleven and K. R. Koedinger, "Limitations of student control: Do students know when they need help?" in *International conference on intelligent tutoring systems*. Springer, 2000, pp. 292–303.